

Resumen: *The Early History of Smalltalk* (Alan C. Kay)

Tomás Felipe Melli

June 19, 2026

Índice

1	Idea central	2
2	Introducción	2
3	1960–1966: Primeras ideas de OOP y el clima de los sesenta	2
3.1	Sketchpad y Simula — “el gran impacto”	3
4	1967–1969: La máquina FLEX, primer intento de PC basada en OOP	4
4.1	Doug Engelbart y NLS	4
5	1970–1972: Xerox PARC — KiddiKomp, MiniCom y Smalltalk-71	6
5.1	Smalltalk-71	6
6	1972–1976: El primer Smalltalk real (-72)	7
6.1	Desarrollo del sistema Smalltalk-72 y aplicaciones	8
6.2	La evolución de Smalltalk-72	9
6.3	El estilo “orientado a objetos”	9
6.4	Smalltalk y los chicos	10
7	1976–1980: El primer Smalltalk moderno (-76)	10
7.1	Herencia	11
7.2	La batalla con Xerox y la visión del futuro	11
7.3	La interfaz de usuario de Smalltalk	11
7.4	Smalltalk-76 (implementación) y la NoteTaker	12
8	1980–1983: La versión de release de Smalltalk (-80)	12
8.1	Coda (reflexiones finales de Kay)	12
9	Apéndices (lo relevante)	13
10	La presentación de Alan Kay (transcripción)	13
11	Comentarios de Adele Goldberg (discussant)	14
12	Sesión de preguntas y respuestas (lo destacado)	15
13	Glosario rápido de tecnologías y personas	15
14	Cierre — 5 ideas para llevarse al final	16

1 Idea central

El paper lo escribió Alan Kay (en ese momento en Apple) para la conferencia *History of Programming Languages II* (HOPL-II, 1993). La sesión la presidió Barbara Ryder y la comentó **Adele Goldberg**, una de las figuras centrales de Smalltalk.

La intuición de la que nace Smalltalk: **todo lo que podemos describir se puede representar por la composición recursiva de un único tipo de “bloque de construcción de comportamiento”**, que esconde dentro de sí mismo su combinación de *estado* y *proceso*, y con el que solo se puede interactuar mediante el **intercambio de mensajes**.

Tres metáforas que Kay repite:

- **Filosófica**: los objetos se parecen a las *mónadas* de Leibniz (entidades cerradas que solo se comunican) y a las *Ideas* de Platón (algunos objetos son idealizaciones —clases— de las que se crean manifestaciones —instancias—). El sistema es completamente autodescriptivo: las clases también son objetos.
- **Biológica**: un objeto es como una **célula** protegida por su membrana, que solo se comunica por mensajes. Un sistema es como “miles y miles de computadoras conectadas por una red muy rápida”.
- **Informática**: Smalltalk es una **recursión sobre la noción misma de computadora**. En vez de dividir la computadora en cosas más débiles (estructuras de datos, procedimientos, funciones), cada objeto es una recursión de toda la capacidad de la computadora.

Frase de **Bob Barton** que es el germen de todo: “*El principio básico del diseño recursivo es hacer que las partes tengan el mismo poder que el todo.*”

2 Introducción

Kay escribe la intro en un avión, usando una notebook de 1992 (su “Dynabook provisorio”): pantalla bitmap de alta resolución, ventanas superpuestas, íconos, dispositivo apuntador, networking, software orientado a objetos. Es —dice— bastante parecido a lo que tenía en mente a fines de los sesenta.

Puntos clave:

- Smalltalk fue parte de una búsqueda más grande que Kay llamó **personal computing**, dentro de ARPA y luego de Xerox PARC. Cita a Goethe: “compartir la emoción del descubrimiento sin intentos vanos de reclamar prioridad”.
- Distingue dos tipos de lenguajes: los de “**aglutinación de features**” (COBOL, PL/1, Ada — hechos por comités) y los de “**cristalización de un estilo**” (LISP, APL, Smalltalk — hechos por una sola persona).
- Kay se reconoce **instigador y diseñador original**, pero insiste en que Smalltalk pertenece más a quienes lo hicieron funcionar, **especialmente Dan Ingalls y Adele Goldberg**. Dan fue la figura central de la implementación.
- El foco del paper está en los eventos que llevaron a Smalltalk-72 y su transición a Smalltalk-76, porque ahí ocurrieron la mayoría de las ideas.

Nota técnica — ARPA / ARPA-IPTO

ARPA (hoy DARPA) era la agencia de investigación del Departamento de Defensa de EEUU. Su oficina IPTO financió en los 60 casi toda la investigación avanzada en computación de EEUU. Su filosofía: **financiar personas y direcciones, no proyectos y metas concretas**. De ahí salieron el tiempo compartido (time-sharing), los gráficos interactivos, las redes y la idea de “simbiosis hombre-máquina” de J.C.R. Licklider. El ARPANet financiado por ahí terminó siendo Internet.

3 1960–1966: Primeras ideas de OOP y el clima de los sesenta

Kay dice que OOP vino de muchas motivaciones, pero dos centrales:

1. **A gran escala**: encontrar un mejor esquema de **módulos** para sistemas complejos, que oculten detalles internos (*information hiding*).
2. **A pequeña escala**: encontrar una versión más flexible de la **asignación** (`:=`), y eventualmente eliminarla.

Las ideas nuevas pasan por etapas: primero “apenas las ves”, luego las notás sin entender su importancia, después las usás operativamente, luego viene una “gran rotación” en la que el patrón se vuelve el centro de un nuevo modo de pensar, y finalmente se vuelve una religión inflexible. (Schopenhauer: una idea nueva primero se denuncia como locura, luego se considera obvia, y al final los que la denunciaron dicen haberla inventado.)

Las primeras veces que Kay “apenas vio” la idea ($\approx$ 1961, Fuerza Aérea)

- **Sistema de archivos del Burroughs 220 (USAF ATC)**: un diseñador anónimo dividió cada archivo en tres partes — (3) los datos reales, (2) los **procedimientos** que saben cómo acceder y actualizar esos datos, y (1) un arreglo de **punteros a los puntos de entrada de esos procedimientos**. Es decir: los datos venían empaquetados con el código que sabía operarlos, accesible solo a través de una interfaz de punteros. Esto es, en embrión, **encapsulamiento e independencia de representación**. Murió cuando se impuso COBOL.
- **Burroughs B5000**: Kay tomó nota de su memoria segmentada, su ejecución por *byte-codes*, el cambio automático de subrutinas y multiproceso, el código puro reentrante (compartible), y sus mecanismos de protección. Vio que el acceso vía la **Program Reference Table (PRT)** se parecía al esquema del 220 (interfaz procedural a un módulo).

Después de la Fuerza Aérea programó recuperación de datos meteorológicos y se interesó por la **simulación** (de una máquina por otra). En 1965, junto a la CDC 6600, leyó un artículo de **Gordon Moore** prediciendo el crecimiento exponencial de los chips (la futura “Ley de Moore”). En ese momento no le pareció relevante.

Nota técnica — Burroughs B5000 (1961)

Mainframe revolucionario diseñado por **Bob Barton**. Adelantado décadas: usaba **byte-codes** (instrucciones compactas interpretadas), **memoria virtual segmentada**, hardware diseñado para compilar lenguajes de alto nivel (ALGOL), **descriptors** (referencias seguras a datos/arreglos/procedimientos con protección tipo “capability”), una **PRT** (tabla con semántica uniforme de nombres) y corrutinas automáticas. Casi nada de esto era común en otras máquinas, que se programaban en assembler crudo. Fue una fuente conceptual enorme para Smalltalk.

Nota técnica — Ley de Moore

Observación de Gordon Moore (1965) de que la cantidad de componentes por chip (y la potencia/costo) mejora exponencialmente con el tiempo. Para Kay fue la justificación de que la computación personal era inevitable: lo que en 1968 era una máquina del tamaño de una habitación, en los 80 cabría en un escritorio.

3.1 Sketchpad y Simula — “el gran impacto”

Kay llega a la Universidad de Utah (1966) “sin saber nada”. El objetivo de Utah dentro de ARPA era resolver el problema de la *hidden line* (líneas ocultas) en gráficos 3D. Dave Evans le da dos cosas para leer/arreglar:

- **Sketchpad** (tesis de Ivan Sutherland, 1963): le voló la cabeza. Tres grandes ideas:
 1. Fue la invención de los **gráficos interactivos modernos**.
 2. Las cosas se describían haciendo un “**dibujo maestro**” (**master**) que producía “**dibujos instancia**” (**instances**) — anticipa clases e instancias.
 3. El control y la dinámica venían de **restricciones (constraints)** gráficas aplicadas a los masters.

Además fue el primero con ventanas de *clipping* y *zoom*.

- **Simula**: junto a otro estudiante desplegó el listado del programa 80 pies por el pasillo y lo recorrió gateando. La parte rara era el **asignador de almacenamiento, que no obedecía una disciplina de pila (stack)** como ALGOL. La revelación: Simula estaba **asignando estructuras parecidas a las instancias de Sketchpad**. Lo que Sketchpad llamaba masters e instances, Simula lo llamaba **activities y processes**. Y Simula era un **lenguaje procedural** para controlar esos objetos, con más flexibilidad que las restricciones.

Este fue el **gran impacto** (“the big hit, and I have not been the same since”). Kay lo conectó con su formación: álgebra abstracta (pocas operaciones que aplican a muchas estructuras) y biología (mecanismos simples controlando procesos complejos; una célula que se diferencia en todos los tipos de células). El sistema de archivos del 220, el B5000, Sketchpad y Simula usaban **la misma idea para propósitos distintos**.

Acá aterriza la frase de **Bob Barton**: “*hacer que las partes tengan el mismo poder que el todo*”. Kay pensó por primera vez en el **todo como la computadora entera**, y se preguntó por qué dividirla en cosas más débiles (datos y procedimientos). ¿Por qué no dividirla en **muchas computadoras chiquitas**, no docenas sino miles, cada una simulando una estructura útil?

Nota técnica — Sketchpad (Ivan Sutherland, 1963)

El primer programa de gráficos interactivos de la historia; corría en la TX-2 del Lincoln Lab. Se dibujaba con un lápiz óptico sobre la pantalla. Introdujo masters/instances, restricciones geométricas, y ventanas con clipping y zoom. Sutherland: “*No sabía que era difícil.*”

Nota técnica — Simula (Nygaard y Dahl, Noruega)

Simula I (1965) y Simula 67 eran extensiones de ALGOL pensadas para **simulación de sistemas**. Introdujeron las **clases** (descripciones que generan múltiples instancias con estado propio y su propio “program counter”), y Simula 67 agregó la **herencia**. Es el ancestro directo más importante de la OOP. A diferencia de ALGOL, sus objetos no vivían en una pila sino en el heap.

Nota técnica — Pila (stack) vs. heap

Los lenguajes tipo ALGOL gestionan la memoria con una **pila**: cuando una función termina, su espacio se libera automáticamente. Los objetos de Simula/Smalltalk necesitan vivir más tiempo que la función que los crea, así que se asignan en el **heap** y requieren gestión de memoria (eventualmente *garbage collection*). Que Simula no usara la pila fue justamente la pista que reveló a Kay que estaba haciendo algo distinto.

4 1967–1969: La máquina FLEX, primer intento de PC basada en OOP

Dave Evans presenta a Kay con **Ed Cheadle**, un ingeniero de hardware que hacía una “maquinita” para profesionales no-informáticos, programable en alto nivel (Cheadle pensaba en BASIC; Kay propuso JOSS, “es más lindo”). Así nace la **máquina FLEX**.

- La máquina era muy chica (máximo 16K palabras de 16 bits) $\Rightarrow$ Simula no entraba.
- JOSS era hermoso por su atención al usuario final, pero lento y sin procedimientos reales ni alcance de variables.
- Adoptaron ideas de EULER (de Wirth): se descartaban tipos, los procedimientos eran objetos de primera clase, y se compilaba a byte-codes tipo B5000, emulables en microcódigo.

Decisiones de diseño importantes de FLEX:

- **Referencias a objetos** como generalización de los *descriptors* del B5000: un descriptor de FLEX tenía dos punteros, uno al **master** del objeto y otro a la **instancia**.
- **Asignación generalizada**: Kay se dio cuenta de que `:=` **no debía ser un operador**, sino una especie de índice que selecciona un comportamiento del objeto. Es decir, `a[55] := 0` debía verse como `a(55, ':=' , 0)`, dejando que el objeto decida qué hacer. Conclusión profunda: “*los objetos son una especie de mapeo cuyos valores son sus comportamientos*”. (Un libro de lógica de **Carnap** sobre definiciones “intensionales” lo ayudó.)
- Corrutinas (de Conway) para suspender y reanudar objetos.
- Una estructura de control **when** (interrupción “blanda” dirigida por eventos), compilada en un árbol que cacheaba resultados parciales — muy parecido a cómo funcionan hoy las **hojas de cálculo**.

4.1 Doug Engelbart y NLS

En 1967 Engelbart visita Utah (“un profeta de dimensiones bíblicas”). Su sistema **NLS** (oN-Line System) buscaba la “**augmentación del intelecto humano**”: hipertexto, gráficos, múltiples paneles, navegación eficiente, **trabajo colaborativo interactivo**. El impacto fue una metáfora convincente de cómo debía ser la computación interactiva.

En este contexto, con la Ley de Moore en la cabeza, Kay hace **el salto**: imaginar la TX-2 (del tamaño de una habitación) o una 6600 puesta sobre un escritorio. En vez de unos pocos miles de mainframes institucionales y usuarios entrenados, habría **millones de máquinas y usuarios personales**, fuera del control institucional. Por lo tanto, el sistema tenía que ser **extensible** y los propios usuarios finales harían la mayor parte del *tailoring* de sus herramientas.

Crítica de Kay a NLS: su interfaz parametrizable forzaba al usuario a estar en un **estado** del que había que salir antes de hacer otra cosa (menús jerárquicos con backtracking). Hacía falta una interfaz más “**plana**”, con una flecha de transición desde cada estado a cualquier otro. Decidió que la idea de **ventana general** de Sketchpad (que ve un mundo virtual más grande) era mejor que los paneles horizontales restringidos.

Nota técnica — NLS y “The Mother of All Demos”

NLS, de Douglas Engelbart (SRI), se presentó en la famosa demo de 1968 que mostró por primera vez el **mouse** (que Engelbart coinventó), hipertexto, edición de texto en pantalla, videoconferencia y trabajo colaborativo.

Nota técnica — JOSS

Sistema interactivo de tiempo compartido de RAND (años 60), célebre por su atención al usuario final y su elegancia. Kay lo cita como un estándar de estética de interacción “no superado”.

Nota técnica — LINC (Wes Clark, 1962)

Considerada la primera computadora personal real (anterior a FLEX). Pequeña, pensada para un único usuario (laboratorios de ciencias de la vida).

Más influencias de 1968–1969 (camino al Dynabook)

- **John Warnock** (futuro fundador de Adobe) propuso que ARPA hiciera reuniones anuales de estudiantes.
- **Marvin Minsky** y las ideas de **Piaget y Papert** sobre educación: replantear el aprendizaje a la luz de la psicología cognitiva. La computación entra como **un nuevo sistema de representación** con metáforas útiles para manejar la complejidad.
- Kay ve el **primer display de panel plano** (de plasma, en Illinois) y calcula — vía Moore — que el silicio de FLEX cabría detrás de un display a fines de los 70.
- En RAND ve **GRAIL**, sucesor gráfico de JOSS, con la **RAND tablet** (primera tableta) y reconocimiento de gestos. Era **manipulación directa, analógico, sin modos (modeless), hermoso**. Kay se da cuenta de que la interfaz de FLEX estaba toda mal.
- Visita a **Seymour Papert** y a quienes hacían **LOGO** con chicos: los chicos programaban de verdad. Insight definitivo: la computación personal no iba a ser un *vehículo* dinámico personal (metáfora de Engelbart), sino un **medio dinámico personal** (“personal dynamic medium”). Como medio, tenía que extenderse al mundo de la infancia.

Nace la imagen del Dynabook: del tamaño de un cuaderno (recordando a Aldus Manutius, que hizo que el libro entrara en las alforjas), con una interfaz tan amigable como JOSS/GRAIL/LOGO pero con el alcance de Simula/FLEX. Kay construyó un **modelo de cartón con perdigones de plomo** para sentir el peso (menos de 2 kg), con teclado y stylus, y networking inalámbrico.

Conferencia de Lenguajes Extensibles (1969): una “guerra religiosa de ideas mal pensadas”. El único que había hecho algo real era **Ned Irons con IMP**: cualquier frase de la gramática podía usarse como cabecera de procedimiento, con definición semántica en términos del lenguaje extendido hasta ese punto. Kay incorporó la idea: que **cada objeto sea un intérprete dirigido por sintaxis de los mensajes que se le envían**. De un golpe, esto unificaba la semántica orientada a objetos con un lenguaje completamente extensible. Imagen mental: **computadoras separadas mandándose pedidos**; cada objeto es un **servidor** que ofrece servicios según su relación con el “servido”.

El año en Stanford y entender LISP

Tras su tesis (*The Reactive Engine*, 1969), Kay va al Stanford AI Lab. Influencias de IA:

- **PLANNER** de Carl Hewitt (base de SHRDLU de Winograd).
- Sistema de **formación de conceptos de Pat Winston**: redes semánticas donde los arcos también se modelaban como redes; base de los *frames* de Minsky.
- **CAL-TSS** (Butler Lampson): un SO basado en **capabilities** muy “orientado a objetos”. El “problema” que los diseñadores no veían: solo ciertas cosas grandes eran “objetos”; las chicas y rápidas no. **Eso había que arreglarlo: que TODO sea objeto.**

Entender LISP de verdad fue el mayor impacto en Stanford. Kay admiró `eval` y `apply`, pero notó **fallas en sus fundamentos**: el lenguaje supuestamente se basaba en funciones, pero sus componentes más importantes (`lambda`, `quote`, `cond`) no eran funciones sino **“formas especiales”** (special forms). Existían **EXPRs** (evalúan sus argumentos) y **FEXPRs** (no los evalúan). Pregunta de Kay: ¿por qué no basar todo en FEXPRs y forzar la evaluación en el lado receptor cuando haga falta? Esto disparó la línea de pensamiento: **“tómalo más difícil y profundo que necesitas hacer, hacelo genial, y construí todo lo más fácil a partir de eso”**. La promesa de LISP era `lambda`; Kay necesitaba algo “más difícil y profundo”: **los objetos debían serlo.**

Nota técnica — LISP (McCarthy, 1960)

Para Kay, “el diseño individual más grande de un lenguaje”. Su característica clave: el **intérprete de LISP está escrito en sí mismo** (metacircular), en aproximadamente una página de código. Esto fue lo que después Kay tomó como desafío para Smalltalk (“definir el lenguaje más poderoso del mundo en una página”). También introdujo el código como estructura de datos y la evaluación respecto de entornos arbitrarios.

Nota técnica — Capabilities (CAL-TSS, GENIE)

En seguridad de sistemas operativos, una *capability* es una referencia infalsificable que otorga ciertos derechos sobre un recurso. Kay vio en esto un modelo de protección de objetos: no alcanza con verificar tipos, hay que **proteger todos los objetos**. De ahí su lema: “*hacelos a todos ciudadanos de primera clase y protegelos a todos*”.

5 1970–1972: Xerox PARC — KiddiKomp, MiniCom y Smalltalk-71

En julio de 1970 Xerox crea **PARC**. **Bob Taylor** (ex ARPA) arma el Computer Science Laboratory. Razón del traslado de talento: la **Enmienda Mansfield** amenazaba con redirigir el financiamiento de ARPA hacia investigación militar directa. Taylor le dice a Kay “seguí tus instintos”.

Kay retoma la KiddiKomp:

- Recuerda otra frase de Barton: “*las buenas ideas no suelen escalar*”. El B5000 no escalaba a una máquina chica; solo los byte-codes lo hacían. Vuelve a mirar la **LINC** de Wes Clark.
- Diseña un lenguaje llamado **SLOGO** (“Simulation LOGO”), para un SONY “tummy trinitron” con display bitmap.
- Se inspira en el **LISP para PDP-1 de Peter Deutsch** (implementado a los 15 años, en solo 2K).
- Insight de Papert: no hace falta hacer mucho para que una computadora sea un “objeto para pensar”, pero lo que se haga tiene que estar bien hecho y aplicarse en profundidad.

En enero de 1971, Taylor atrae a PARC a gran parte de la **Berkeley Computer Corp.**: Butler Lampson, Chuck Thacker, Peter Deutsch, Jim Mitchell, entre otros. También llega **Bill English**, coinventor del mouse. Conflicto con Xerox: el grupo quería una PDP-10 (de DEC, competidor) $\Rightarrow$ terminaron construyendo su propia emulación, la **MAXC**, que por primera vez usó chips de memoria integrada (¡1K bits!) en vez de núcleos de ferrita.

Anécdota de la “belleza de LISP”: Allen Newell visita PARC con su teoría del pensamiento jerárquico. Le dan un problema (índices impares seguidos de pares). Newell, con su lenguaje tipo IPL-V (punteros explícitos), batalla más de 30 minutos. Kay lo escribe en LISP en segundos:

```
1 oddsEvens(x) = append(odds(x), evens(x))
```

Lección: “*el punto de vista vale 80 puntos de IQ*”. No era más inteligente, tenía mejor herramienta mental.

Conflicto con Don Pendery (el “planificador” de Xerox que solo quería “tendencias”): Kay le contesta con su frase famosa: “**La mejor manera de predecir el futuro es inventarlo.**” De ahí los “Pendery Papers”.

Kay arma su grupo, el **Learning Research Group (LRG)** — nombre deliberadamente vago. Contrataba solo gente “a la que se le iluminaban los ojos” con la idea de la notebook. Cuando alguien no sabía qué hacer, señalaba el modelo de cartón: “*Avanzá eso.*”

5.1 Smalltalk-71

En el verano del 71, Kay refina la KiddiKomp en **miniCOM** y un lenguaje que **ahora llama “Smalltalk”** — como en “programar debería ser una cuestión de *small talk* (charla)”. El nombre era a propósito modesto, en reacción a los sistemas con nombres de dioses (Zeus, Odin, Thor) que “no hacían nada”.

Smalltalk-71 estaba muy influido por FLEX, PLANNER, LOGO y META II. Era una especie de **parser con object-attachment que ejecutaba tokens directamente** (pattern matching). Kay quería **control total sobre lo que se pasa en un envío de mensaje**: específicamente, **cuándo y en qué entorno** se evalúan las expresiones.

Ejemplos de Smalltalk-71:

```
1 to 'factorial' 0 is 1
2 to 'factorial' :n do 'n*factorial n-1'
3 to :e 'is-member-of :group
4     do 'if e = first of group then T
5     else e is-member-of rest of group'
```

Nota técnica — META II (Schorre, 1963)

Un “compiler-compiler”: un lenguaje para escribir compiladores definiendo gramáticas. De ahí venían las “raras convenciones de comillas” de Smalltalk-71.

Nota técnica — Bitmap display

Pantalla donde cada píxel está mapeado a un bit (o varios) en memoria, permitiendo dibujar gráficos arbitrarios y texto con tipografías variables. Antes lo común eran las “glass teletypes” (solo caracteres fijos). El bitmap es la base de toda interfaz gráfica moderna.

Tesis de Dave Fisher y “diseño reflexivo”

La tesis de **Dave Fisher** (CMU, 1970) sobre síntesis de estructuras de control influyó mucho: generalizar los enlaces de ALGOL-60 (dinámico para subrutinas, estático para estado global) para simular muchísimas estructuras de control. Anticipa el **diseño reflexivo** y la **evaluación perezosa (lazy evaluation)**.

Conclusión de Kay: para “hacer LISP bien” u “OOP bien” alcanzaba con manejar la mecánica de las **invocaciones entre módulos** sin preocuparse por los detalles internos. La diferencia entre LISP y OOP sería solo **qué pueden contener los módulos**. Hacía falta una **referencia universal de objeto** (à la B5000/LISP) y una **estructura portadora de mensajes** (un “messenger”, generalización de un stack frame, con campos GLOBAL, SENDER, RECEIVER, REPLY-STYLE, STATUS, REPLY, OPERATION SELECTOR, parámetros).

Reflexión sobre la belleza: viene de la sinergia entre **parsimonia, generalidad, esclarecimiento y finura** (ej.: el teorema de Pitágoras). Un buen sistema necesita “alta pendiente”: buena relación entre lo interesante que es y la complejidad para expresarlo. Metáfora favorita: el **óvulo fertilizado** que se transforma en un organismo complejo — elegancia y practicidad a la vez, como la membrana celular que presenta una interfaz uniforme al mundo.

El insight de las ventanas superpuestas: a Kay le preocupaba el tamaño del display bitmap. En la ducha se le ocurre que las ventanas tipo FLEX podían aparecer como **documentos superpuestos en un escritorio**, donde la que se refresca sube al tope de la pila.

El hardware de soporte: Bill English y Butler Lampson especifican un generador de caracteres experimental (Roger Bates) para las terminales POLOS. Gary Starkweather hizo funcionar la **primera impresora láser** (“SLOT machine”, 500 píxeles por pulgada). Ben Laws hizo un editor de fuentes.

Nota técnica — NOVA

Minicomputadora de Data General (años 60–70). El “10 times faster NOVA” fue uno de los objetivos del ALTO.

Nota técnica — Microcódigo / emulación

Las instrucciones de byte-code de Smalltalk no corrían directamente en el hardware: un programa de **microcódigo** (más cercano al hardware) las interpretaba/emulaba. Permitía implementar funciones de alto nivel sin hardware dedicado. Butler Lampson sugirió un “Huffman pobre”: mapear distintos significados a distintas partes del espacio de 256 valores de un byte.

Tensión con Piaget/Bruner y giro hacia lo icónico

Tras leer a **Piaget y Bruner**, Kay se preocupa de que el enfoque puramente simbólico (FLEX, LOGO, Smalltalk) sea difícil para los chicos. Educadores admirados (Montessori, Holt, Suzuki) pedían un enfoque más **figurativo/icónico**. GRAIL no servía (imágenes para flowcharts), pero **AMBIT-G** de Rovner (una especie de SNOBOL visual) prometía. Plan: un sistema real con OOP, ventanas, pintura, música, animación y “**programación icónica**”.

Lema: “**Las cosas simples deberían ser simples, las cosas complejas deberían ser posibles.**”

6 1972–1976: El primer Smalltalk real (-72)

Dos “apuestas” cambian todo en septiembre de 1972:

Apuesta 1 — el hardware: Butler Lampson y Chuck Thacker le ofrecen a Kay construirle su “maquineta” (tenía ~\$230K). Butler quería un “PDP-10 de \$500”, Chuck una “NOVA 10 veces más rápida”, Kay una “kiddicomp”. Chuck había apostado hacer la máquina en tres meses. Nace el **ALTO** (el “Interim Dynabook”).

Apuesta 2 — el lenguaje: En una charla de pasillo, Ted Kaehler y Dan Ingalls preguntan cuán grande tendría que ser un lenguaje para tener gran poder. Kay afirma que se podía definir “**el lenguaje más poderoso del mundo en una página de código**”. Le dicen “poné lo que prometés o callate”. La base era el intérprete metacircular de LISP de McCarthy.

Fue más difícil de lo esperado: (1) quería un intérprete no-recursivo (más “real”); (2) el entrelazado del “parseo” con la recepción de mensajes obligaba a reentrar antes que LISP; (3) no tenía claro cómo debían funcionar **send** y **receive**. Igual logró una versión que funcionaba. Pocos días después, **Dan Ingalls la mostró corriendo en la NOVA** (¡codificada en BASIC!). Frase de Dan: “*Lo hacés y ya está.*” Evaluaba 3+4 **muy lentamente** (“glacial”), pero el resultado siempre era 7.

Esto fue **Smalltalk-72**. Dan, a lo largo de 10 años, hizo al menos 80 releases mayores. En noviembre Kay presentó las ideas en el MIT AI Lab, lo que llevó al modelo de **Actores** de Carl Hewitt.

El ALTO (“Bilbo”): Chuck Thacker empezó el 22/11/1972 y en abril de 1973 estaba listo. Display de ~500.000 píxeles (606×808), ~6 MIPS, 96 KB, todo en 160 chips MSI. Característica genial: **tasking de “overhead cero”** con 16 program counters, arquitectura de corrutinas. La primera imagen fue el **Cookie Monster** de los Muppets. Se construyeron unos 2000 ALTOS.

Conflictos con Xerox y Rolling Stone: el ejecutivo “X” se opuso a construir más miniCOMs. Un artículo de Stewart Brand en *Rolling Stone* (1972) causó furor en la central de Xerox: credenciales obligatorias y restricción de publicaciones — desastroso para el LRG, que necesitaba compartir ideas.

Las 6 ideas principales de Smalltalk-72

Las primeras 3 son “objetos vistos desde afuera” (no cambiaron nunca); las últimas 3 son “desde adentro” (se modificaron en cada versión):

1. **Todo es un objeto.**
2. **Los objetos se comunican mandando y recibiendo mensajes** (en términos de objetos).
3. **Los objetos tienen su propia memoria** (en términos de objetos).
4. **Todo objeto es instancia de una clase** (que debe ser un objeto).
5. **La clase guarda el comportamiento compartido** de sus instancias (objetos en una lista de programa).
6. **Para evaluar una lista de programa, el control se pasa al primer objeto** y el resto se trata como su mensaje.

De (1) y (4) se sigue que **las clases son objetos y deben ser instancias de sí mismas**. De (6) se sigue una **sintaxis universal tipo LISP**, con el objeto receptor primero, seguido del mensaje:

```
1 receptor | mensaje
2 c         | i <- d*e
```

Expresiones “simples” como a+b: Kay las interpreta también como `receptor | mensaje (3 | +4)`. Esto lleva al **polimorfismo**: buscar comportamientos genéricos para los símbolos de mensaje (+ puede concatenar strings o sumar matrices según el objeto).

Protección total: como el control se pasa a la clase **antes** de considerar el resto del mensaje, la clase puede decidir no recibir. Los objetos de Smalltalk-72 son “brillantes” (shiny) e impenetrables.

Función vs. clase: Smalltalk-72 mantenía de Smalltalk-71 la mezcla de ideas de función y clase. Se podía producir una instancia (una especie de *closure*) con el objeto ISNEW. **Toda la idea de OOP es definir todo intensionalmente.** A Kay no le gustaba la sintaxis (demasiados paréntesis); quería algo más plano (lo veríamos en Smalltalk-76).

Nota técnica — Polimorfismo

Capacidad de que un mismo nombre de mensaje (ej. `+`, `print`) produzca comportamientos distintos según la clase del objeto receptor. En Smalltalk cada objeto decide qué hace con el mensaje que recibe.

Nota técnica — Intensional vs. extensional

Definir algo **extensionalmente** es listar/calcular sus elementos directamente; **intensionalmente** es definir la propiedad/-comportamiento que los caracteriza. OOP busca definir todo intensionalmente (vía comportamiento del objeto), no manipulando su estado interno.

6.1 Desarrollo del sistema Smalltalk-72 y aplicaciones

El intérprete en el ALTO no era veloz (“majestuoso”, según Butler), pero era fácil de cambiar. Aplicaciones (muchas en pocas páginas de código):

- **Ventanas superpuestas** (con Diana Merry) y un primer **bitblt**. Usaban las convenciones de **GRAIL** (esquinas sensibles para mover, redimensionar, clonar, cerrar).
- **Tortugas LOGO orientadas a objetos** (Ted Kaehler); Dan creó tortugas “comandante”.
- Editor estructurado de código (John Shoch).
- **miniMOUSE** (Larry Tesler): el primer editor **WYSIWYG** sin modos de PARC.
- Documentos **multimedia** (Steve Weyer y otros): cada componente maneja su propia edición.
- **Findit** (Weyer y Kay): recuperación por ejemplo, usado años por la biblioteca de PARC.
- Música: síntesis por muestreo (12 voces en tiempo real), **TWANG** (Kaehler), síntesis FM en tiempo real (Saunders) y **OPUS** (Chris Jeffers), captura de partituras sin ritmo metronómico.

- Animación: **Steve Purcell** logró “tasas Disney” (10–15 fps); de ahí **Shazam**.
- **PYGMALION** (tesis de Dave Smith): programación icónica/“por demostración”; el programa más grande de Smalltalk-72 (~20 páginas). Origen de los “programming by example”.
- **Simpula**: versión simple del scheduling de SIMULA. Base del futuro **Smalltalk Sim-kit**.

Nota técnica — bitblt (Bit Block Transfer)

Operación que copia/combina bloques rectangulares de píxeles en memoria de pantalla. Es el operador 2D fundamental para mover ventanas, dibujar fuentes, etc. Diana Merry hizo una versión temprana; luego se metió en microcódigo. Sigue siendo central en gráficos.

Nota técnica — WYSIWYG

“What You See Is What You Get”: lo que ves en pantalla es lo que se imprime. miniMOUSE de Tesler fue pionero. Tesler luego llevó estas ideas a Apple.

6.2 La evolución de Smalltalk-72

Smalltalk-74 (alias “FastTalk”): un objeto “messenger” real, diccionarios de mensajes para clases (paso hacia clases-objeto reales), bitblt en microcódigo, mejor interfaz de ventanas. Dave Robson ayudó a formular una semántica oficial.

La gran incorporación: **OOZE** (Object-Oriented Zoned Environment), el sistema de **memoria virtual** que sirvió a Smalltalk-74 y especialmente a Smalltalk-76. El ALTO era chico (128–256K). OOZE intercambiaba (swap) objetos individuales:

- Gastar un pequeño porcentaje de tiempo “purgando” objetos sucios para mantener memoria limpia (insight de Lampson en GENIE); mecanismo de decisión estocástico (de FLEX) para decidir qué descartar.
- Integridad de punteros: una “**transaction**” completa (idea de Butler) que garantizaba recuperación sin importar cuándo se colgara el sistema (“protección contra rayos cósmicos”).
- Una **Resident Object Table (ROT)** como único lugar con direcciones de máquina, y un esquema de **checkpointing** con imagen recuperable de pocos segundos de antigüedad.
- Manejaba ~65K objetos en solo 80KB de memoria de trabajo.

Nota técnica — Memoria virtual / swapping

Técnica para usar el disco como extensión de la RAM, moviendo (swap) datos entre disco y memoria. OOZE lo hacía a nivel de objetos individuales. El *garbage collection* (recolección de basura) automática de objetos no usados es parte de este mundo.

6.3 El estilo “orientado a objetos”

Sección clave. Kay distingue OOP de los “**tipos de datos abstractos**” (**abstract data types**):

- Un ADT (como la definición de “par LISP”) preserva el “acceso a campos” y “rebinding de campos” — sigue siendo una **estructura de datos**.
- El mundo académico veía a Simula como vehículo para ADTs (y eso fue al backbone de ADA, con el ubicuo ejemplo de la “pila”). **Kay se horrorizó: lo que Simula le dijo fue que se podían reemplazar los bindings y la asignación por metas (goals).**
- Lo último que querés es que un programador toque el estado interno. Los objetos deben presentarse como **sitios de comportamientos de alto nivel**, apropiados como componentes dinámicos.

Crítica de Kay (1993): mucho de lo que se llama “OOP” hoy es **programación al viejo estilo con constructos más sofisticados**.

¿De dónde viene la eficiencia especial de OOP? Cuatro técnicas usadas juntas:

1. **Estado persistente** (persistent state),
2. **Polimorfismo**,
3. **Instanciación**,

4. Métodos-como-metas (methods-as-goals).

Ninguna requiere un lenguaje OO (ALGOL 68 casi se puede forzar a este estilo); un OOP solo **enfoca la mente del diseñador**. Hacer el encapsulamiento bien es un compromiso no solo con abstraer el estado, sino con **eliminar las metáforas orientadas al estado**.

Principio más importante (derivado de los SO): cuando le das a alguien una estructura, rara vez querés que tenga privilegios ilimitados. Type-matching no alcanza. **Hacelos a todos ciudadanos de primera clase y protégelos a todos**.

El menor tamaño de un buen sistema OOP viene del **“bang por línea de código”**. Idea final: **OOP es una estrategia de late binding (ligadura tardía)** de muchas cosas. *“Los programadores humanos no son máquinas de Turing — y cuanto menos requieran sus sistemas técnicas de máquina de Turing, mejor.”*

Nota técnica — Late binding (ligadura tardía)

Decidir lo más tarde posible (idealmente en tiempo de ejecución) detalles como qué método se ejecuta, dónde está algo, cómo se representa. Es lo opuesto a “compilar todo rígidamente de antemano”. Kay considera el late binding el corazón de OOP.

6.4 Smalltalk y los chicos

Desde el verano del 73 empiezan los experimentos con chicos. Se suman **Adele Goldberg** y Steve Weyer.

- Primeros experimentos: imitar tortugas de LOGO. Resultado decepcionante (“efectos de superficie”). Kay sentía que el contenido debía ser la **creación de herramientas interactivas por los chicos**.
- **El “Joe Book” de Adele**: enseñar Smalltalk con ejemplos de programas serios (influido por Minsky). Se creaban instancias de la clase box y se les mandaban mensajes (**draw, undraw, turn, grow, ISNEW**).
- **Proyectos de chicos que fueron herramientas reales**: sistema de pintura de Marion Goldeen (12), sistema de ilustración OO de Susan Hamet (12, parecido al futuro MacDraw), captura de partituras de Bruce Horn (15), diseño de circuitos de Steve Putz (15).
- **Reality check (“early success syndrome”)**: los éxitos eran reales pero no tan generales. Veían el “fenómeno hacker”: un 5% se zambulle naturalmente; el ~80% puede aprenderlo pero no le resulta natural.
- **El problema era el diseño, no la mecánica**. Kay enseñó Smalltalk a 20 adultos no programadores; chocaron con un rolodex que parecía simple pero tenía **17** “ideas no obvias”.

La conexión con la alfabetización (literacy): aprender a leer y escribir no alcanza; hay una **literatura** que organiza ideas. Adele inventó las **design templates**: un intermediario entre las ideas vagas y el código, para descomponer un problema en clases y mensajes sin preocuparse aún por cómo funciona cada método.

- Empujaron la **herencia** para que novatos construyeran sobre frameworks de expertos, pero resultó **muy difícil para novatos (e incluso profesionales)**.
- Conclusión: programar, para el “80%”, se parece más a la **escritura**: hay que aprenderlo gradualmente durante años.

Reflexión más amplia de Kay: ¿deberíamos siquiera enseñar programación? No ve influencia clara en la capacidad general de pensar. “La música no está en el piano.” Las herramientas dan camino y contexto, pero **ninguna herramienta contiene ni dispensa el esclarecimiento**. Pavese: *“para conocer el mundo debemos construirlo”*. Sobre **fluidez y metáfora**: la fluidez hace desaparecer la interpretación de las representaciones; el “truco” de las artes liberales es volverse fluido y profundo mientras se construyen relaciones con otros conocimientos. Su **alfabetización del siglo XX** incluye oír de una enfermedad contagiosa y entender al instante la relación exponencial desastrosa — e incluso simularla en la computadora personal.

7 1976–1980: El primer Smalltalk moderno (-76)

A fines de 1975 Kay siente que pierden el equilibrio. En enero de 1976 lleva al grupo a un retiro: **“Let’s Burn Our Disk Packs”**. Su aforismo: *“ningún organismo biológico puede vivir en sus propios desechos”*. Todos coincidían en que el poder de Smalltalk no alcanzaba sus aspiraciones, pero los grad students querían un Smalltalk mejor y más rápido.

Resultado: Kay empieza a diseñar la **NoteTaker** y **Dan empieza a diseñar Smalltalk-76**. La cohesión de los primeros cuatro años no volvió a cuajar. Sensación McLuhaniana: *“una vez que damos forma a las herramientas, ellas nos dan forma a nosotros”*. Los paradigmas fuertes (LISP, Smalltalk) **“se comen a sus crías”**: una aplicación se parece al sistema, no a una idea nueva. Kay en 1975 **no veía un lenguaje de usuario final**.

Smalltalk-76 (Dan Ingalls): el primer gran cambio fue **eliminar el dualismo función/clase** en favor de una definición **completamente intensional**, con cada pieza de código como método intrínseco. Demanda de **herencia real**. Se hizo realidad que **“todo es un objeto”**, incluyendo los objetos internos (registros de activación). Dan creó una sintaxis combinada **keyword/operador**, legible sin ambigüedad por humanos y máquina, con un compilador de byte-codes tipo FLEX que corría **hasta 180 veces más rápido** que el intérprete directo anterior.

7.1 Herencia

- **Simula-I**: no tenía ni clases-como-objetos ni herencia.
- **Simula-67**: agregó herencia como generalización de <block> de ALGOL-60. Gran idea, con desventajas: rigidez de tipos, necesidad de calificar tipos, **un solo camino de herencia**, dificultad con desarrollo interactivo/compilación incremental.
- Por eso Kay **dejó la herencia afuera de Smalltalk-72** (sabiendo que se podía simular con la flexibilidad tipo LISP). **Larry Tesler** usó “slot inheritance” (hoy **herencia por delegación**). Bobrow y Winograd diseñaban KRL con herencia múltiple (*perspectives*).
- En **Smalltalk-76**, Dan llegó a un esquema **tipo Simula en su semántica pero modificable en caliente**. **Alan Borning** usó herencia múltiple en **Thinglab**, pero ningún esquema limpio superó el diseño de Dan.

Nota técnica — Herencia / herencia por delegación

Mecanismo por el cual una subclase hereda comportamiento de una superclase. La **delegación** es una variante donde un objeto reenvía mensajes que no sabe manejar a otro objeto “padre”. La herencia múltiple (heredar de varias superclases) trae el problema de conflictos de nombres de métodos.

7.2 La batalla con Xerox y la visión del futuro

Había ~500 ALTOs conectados por **Ethernet** a impresoras láser y servidores de archivos. El memo de Kay “A Simple Vision of the Future” (actualización del Pendery Paper de 1971) predice con precisión: en los 90 habrá millones de PCs del tamaño de un cuaderno, con displays planos, llamados Dynabooks; precio como un TV color; la mayoría se regalarán porque se venderá el **contenido**, no el contenedor; conectadas a “utilidades de información” globales. Componentes críticos: **Personal Computers + Communications Links + Information Utilities**. **Todo el contenido y la función se hacen en software**.

La decisión fatal de Xerox (agosto 1976): Chuck Thacker diseñó el **ALTO III** (comercializable). Xerox decidió **no llevarlo al mercado**. Kay: en 1992 el mercado de PCs/workstations era de \$90 mil millones; **la empresa más exitosa de la era —Microsoft— no es de hardware sino de software**.

Nota técnica — Ethernet y el ALTO

Ethernet (LAN por conmutación de paquetes) fue inventado en PARC por Bob Metcalfe. Los ALTOs en red con impresoras láser y servidores de archivos eran, en 1976, el prototipo completo de la oficina informática moderna — una década antes de que el mundo lo tuviera.

7.3 La interfaz de usuario de Smalltalk

Todos los elementos ya existían en los sesenta (Lincoln Labs y RAND, financiados por ARPA). El gran cambio fue que **el foco del LRG era en los chicos** y en el **aprendizaje**: una rotación de 90 grados del propósito de la interfaz — de **“acceso a la funcionalidad”** a **“entorno donde los usuarios aprenden haciendo”**.

Síntesis de principios de diseño de la UI:

- Entorno aparentemente libre donde la exploración causa las secuencias deseadas (**Montessori**);
- aprendizaje kinestésico, icónico y simbólico — *“hacer con imágenes hace símbolos”* (**Piaget y Bruner**);
- el usuario **nunca queda atrapado en un modo** (**GRAIL**);
- la magia embebida en lo familiar (**Negroponte**);
- un **espejo amplificador de la propia inteligencia** (**Coleridge**).

La consolidación se atribuye a **Dan Ingalls**. **Dave Smith** diseñó SmallStar, prototipo de la interfaz icónica del Xerox **Star**.

7.4 Smalltalk-76 (implementación) y la NoteTaker

Dan, Dave Robson, Ted Kaehler y Diana Merry implementaron el sistema desde cero en **solo 7 meses**. ~50 clases en ~180 páginas de código (SO, archivos, impresión, Ethernet, ventanas, editores, gráficos, pintura), más los **browsers** de Tesler y los contextos dinámicos de debugging. *“Todos los Smalltalks posteriores se parecen mucho a esta concepción.”*

Sintaxis de Smalltalk-76 (clases Window/DocWindow): el código se expresa como **metas para otros objetos**. `:` = keyword por valor; `^` = “send back”; `=>` = “entonces”; `super` = delegar a la superclase.

Primera prueba real (enero 1978): los 10 ejecutivos top de Xerox visitan PARC. El LRG decide **no** enseñarles Smalltalk-76, sino crear en 2 meses el **Smalltalk SimKit** (basado en Simpula) para adultos no expertos. Adele fue la líder de diseño. **9 de 10 ejecutivos completaron una simulación** relevante (uno modeló una línea de producción y descubrió una falla seria). Un VP: *“así que finalmente llegamos a esto”*.

Otro sistema importante: **Thinglab** de Alan Borning (1979), primer intento serio de ir más allá de Sketchpad, con **restricciones (constraints)**. Mostró cómo pasar del estilo “push” al estilo “pull”. También descubrieron que los **“prototipos”** eran más hospitalarios que las clases.

La NoteTaker (Intel 8086): en 1978 necesitaban tres 8086 (intérprete, gráficos, I/O). Dan hizo correr Smalltalk-76 en 256K (system tracer de Kaehler; apareció la tabla de objetos indexada, luego usada en Smalltalk-80). El kernel quedó en 6K de código 8086 y corría ~2 veces más rápido que en el ALTO. La NoteTaker **corrió a 35.000 pies en un avión** con baterías. Se construyeron ~10.

La visita de Steve Jobs (1979): Jobs, Jeff Raskin y otros de Apple (proyecto Lisa) visitan PARC. Usaron el **Dorado** (un “hermano mayor” veloz del ALTO). Momento famoso: Jobs pidió scroll continuo; **Dan lo cambió en menos de un minuto**, shockeando a los visitantes. Jobs intentó comprar la tecnología, pero Xerox ni la cedió ni la desarrolló.

Nota técnica — Xerox Star, Apple Lisa y Macintosh

El Xerox Star (1981) fue el primer producto comercial con GUI (descendiente del ALTO), pero caro y débil comercialmente. La visita de Apple a PARC influyó directamente en la **Lisa** y luego la **Macintosh** (1984), que llevaron la GUI al mercado masivo. Larry Tesler dejó Xerox por Apple en 1980.

Nota técnica — Dorado

Sucesor mucho más rápido del ALTO (uno de los “D machines”). En la demo a Apple se usó porque el ALTO ya era lento. Kay opina que el atraso del primer “D machine” dañó a Smalltalk: debieron seguir construyendo máquinas más rápidas hasta entender bien la OOP, en vez de regresar (en Smalltalk-76) hacia formas tipo Simula por eficiencia.

8 1980–1983: La versión de release de Smalltalk (-80)

Dan: *“La decisión de no continuar la NoteTaker dio motivación para liberar Smalltalk ampliamente.”* Pero a Kay le entristecía que **ningún chico había programado en Smalltalk desde Smalltalk-76**. Xerox y PARC pensaban en “workstations”; Kay seguía queriendo “playstations”. Larry Tesler se fue a Apple; Kay se tomó un sabático.

Adele condujo la documentación y el release. Cambios a la NoteTaker Smalltalk-78:

- El más irónico: volver las fuentes custom a **ASCII estándar** (Deutsch: “esencial para la aceptación del sistema en el mundo”).
- Hacer los *blocks* más parecidos a expresiones lambda.
- La más desconcertante para Kay: las **metaclases** (Deutsch: “más confusas que valiosas”). Kay opina que Smalltalk entró en la fase final: **una manera de hacer las cosas que se canoniza en una estructura de creencias inflexible**.

8.1 Coda (reflexiones finales de Kay)

Barton: *“Los programadores de sistemas son los altos sacerdotes de un culto bajo.”*

Hardware como “software cristalizado tempranamente”: gran parte del progreso del software ha sido encontrar formas de **late-bind**, y luego convencer a los fabricantes de construirlas en hardware. Ejemplos históricos de cosas “late-bindeadas”: RAM (programas/parámetros antes cableados), registros índice, base/bounds registers, relocación de segmentos, MMUs de paginación, recursión.

OOP desde la perspectiva del late binding: es una técnica comprensiva para ligar tardíamente todo lo posible. *“El arte del wrap es el arte del trap.”* El punto de OOP: **no tener que preocuparse por lo que hay dentro de un objeto**. Anticipa el **networking omnipresente**: objetos como **agentes activos** que viajan por las redes y se configuran por **“acoplamiento inferencial”** (inferential docking). Menciona el **metaobject protocol** (Kiczales), Prolog (no necesita bindings a valores) y Eurisko (“programación oportunista”).

Teoría irónica del “sunspot”: avances cada ~ 11 años. Código de máquina (1950) $\rightarrow$ FORTRAN (1956) $\rightarrow$ ALGOL-60 (1961) $\rightarrow$ SIMULA (1966) $\rightarrow$ **Smalltalk (1972)** $\rightarrow$ Eurisko (1978). Pero 1983 vino sin “la cosa nueva”. Kay culpa a la **comercialización masiva** de la computación personal, que “ahogó” el trabajo de universidades y labs.

“Vandalismo inverso”: la tecnología se volvió “demasiado fácil”; los diseños son triviales (“*hacer cosas porque se puede*”). El antídoto: enorme insatisfacción con los propios diseños, usando toda la historia del arte humano como estándar, pero desacoplada de la autoestima.

Cierra dejando la historia en 1981 (*Byte*) y 1983 (libros de Adele y Robson, release oficial). Pregunta final: “¿*dónde están los Dans y Adeles de los 80 y 90 que nos llevarán a la siguiente etapa?*”

9 Apéndices (lo relevante)

- **Apéndice I — Memo de la PC (“KiddiKomputer”, mayo 1972)**: cuatro proyectos educativos: (a) enseñar a *pensar* (Papert); (b) enseñar *modelos* vía simulación (música y programación); (c) habilidades de *interfaz* (ver/oír, lectura icónica+audible); (d) ver qué hacen los chicos “extraoficialmente” con “demons”.
- **Apéndice II — Diseño del intérprete**: cómo Kay ganó la apuesta. Muchos detalles del esquema de McCarthy se “fineseaban” con objetos que manejan recepción parcial de mensajes. La interpretación va de **izquierda a derecha**: el primer elemento se evalúa en el objeto receptor, todo lo que sigue es el mensaje. “*Un send no es como un cartero entregando una carta, sino entregando un aviso de dónde está — el receptor decide qué hacer.*”
- **Apéndice III — Acknowledgments (1973)**: influencias declaradas: Papert/MIT (filosofía); el Dynabook como “ahijado” de la LINC y descendiente del FLEX; el ALTO de Thacker y McCreight; “SMALLTALK es definitivamente LISP-like”; B5000, las SIMULAs, FLEX y el **CDL de Dave Fisher** (fuente principal de la semántica de control). La charla de Barton en Alta (1968) fue “la verdadera génesis”. Principio: **hacer que las PARTES tengan el mismo poder que el TODO**.
- **Apéndice IV — Event Driven Loop**: clase `event` y estructura `until` por eventos. “Eventualmente nos calmamos y nos enfocamos en estructuras más simples y de mayor poder.”
- **Apéndice V — Estructuras internas de Smalltalk-76**: un método compilado en byte-codes de la clase `Rectangle`. “El esquema general se remonta al B5000 y al FLEX, pero está mucho más refinado.”
- **Apéndice VI — Documentación**: “paisaje general de los sesenta” con las tecnologías que aportaron a OOP. Lo común: **interés en encontrar estructuras simples y generales para “fintar” dificultades**, en vez de producir grandes cantidades de código.

10 La presentación de Alan Kay (transcripción)

Charla sobre “el romance del diseño”: cómo se hace la innovación radical. En PARC en 1974 había, sin un plan maestro: la primera PC/workstation, ventanas superpuestas, OOP moderna, impresora láser y LAN por paquetes — hechas por grupos distintos de gente amigable. Filosofía ARPA: **financiar personas y direcciones**.

Marcos conceptuales:

- **Koestler — “Bisociación”** (*The Act of Creation*): pensamos relativo a contextos (paradigmas, à la Kuhn). “*Las ideas terríficas se esconden detrás de las buenas.*” La creación se parece a un chiste. Reacciones: chiste = “ja”, ciencia = “ajá”, arte = “aah”.
- **McLuhan**: “*No sé quién descubrió el agua, pero no fue un pez.*” No vemos el medio/creencias en que estamos inmersos (la “pecera”). La cultura es una mejor pecera.
- **La rana**: su ojo no le dice mucho al cerebro; rodeada de moscas paralizadas se muere de hambre. *Las ideas son como esas moscas: están enfrente pero no las vemos.* “Solo podés aprender a ver cuando te das cuenta de que sos ciego.”

Gente clave:

- **Financiadores de ARPA** (Licklider, Sutherland 26, Taylor 34, Roberts 31).
- **Dave Evans**: trataba a los grad students como colegas. “Siempre dejá un bit extra” (preocupate por las condiciones de excepción, no por el camino obvio).
- Contexto de Utah: **3D a gran escala**; necesitabas un **principio de arquitectura**. Crítica: enseñar **algoritmos como primer curso** es lo peor.

- **Simula como punto de cruce:** el más grande desde LISP. Kay vio en Simula **tejidos/células**: lo transformó de pensar las computadoras como mecanismos a pensarlas como lugares para hacer **organismos tipo biológico**. “*Después de Simula, los procedimientos me parecían la peor manera posible de hacer cualquier cosa.*”
- **Bob Barton:** “*Mi trabajo es desengañarlos de todas las nociones queridas que trajeron a este aula.*”
- **Ivan Sutherland:** “*No sabía que era difícil.*”
- **Marvin Minsky:** “*Tenés que formar el hábito de no tener razón por mucho tiempo.*” Formó “*tri-stinciones*”.
- **El mejor libro sobre diseño de sistemas complejos:** *The Federalist Papers*.

En el **video** mostró: los ALTOs tempranos; cómo Adele enseñaba con “Joe”; el sistema de pintura de Marion Goldeen (12); el tipo MacDraw de Susan Hamet (12); la animación de Steve Purcell; Shazam; el editor de timbres FM tocando una fuga de Bach; el SimKit; y **Playground**, donde chicos programaban un pez payaso con ~25 procesos paralelos. Cierre: “*las artes de la computadora son la imitación de la creación misma.*”

11 Comentarios de Adele Goldberg (discussant)

Adele resume Smalltalk no como la evolución de un lenguaje sino como **una historia de cultura, organización y visión**. Lecciones de Kay:

1. “**Elegí un sueño, hacelo realidad.**” (Kay: “*No predigo el futuro, lo hago.*”) El sueño: **las computadoras no son interesantes; lo interesante es lo que la gente puede hacer y acceder con ellas.**
2. “**Contratá gente que no sepa que lo que querés que hagan es difícil.**”
3. “**Si no compartís tus resultados, podés quemar los discos y empezar de nuevo.**” (Aunque violaba: “*No tengas clientes; esperan soporte.*”)
4. “**Tené cuidado con lo que deseás, porque lo vas a conseguir.**”

Aportes de Adele:

- Trabajar con chicos introdujo la **pedagogía como objetivo de calidad**. Vieron el problema **no como crear un lenguaje, sino como crear un sistema para crear sistemas** (legibles por quien entienda el dominio). La **legibilidad** se volvió crítica.
- Insistió (sin éxito total) en que **enseñar Smalltalk a chicos no implicaba resolver la aprendibilidad para adultos**; las capacidades de **simulación/modelado** son el determinante clave.

Historia del release y frustración con Xerox: el “Smalltalk Hall of Shame” (1979, primer miembro: el jefe de PARC que dijo que a nadie le interesaba cargar computadoras); el proyecto **Monk** (no financiado); **Steve Jobs** sí entendió la idea. Decidieron compartir publicando **el sistema Smalltalk-80 mismo**.

Release de **Smalltalk-80** (multivendor): empresas que hacían hardware con equipos de software in-house — **Apple, Hewlett-Packard, DEC y Tektronix**. Tres partes: rediseñar subsistema por subsistema (gracias a poder convertir todos los objetos de una clase en otra; Kaehler: “botar un barco mientras se construye”); especificación formal de la **máquina virtual** (libro *Bits of Wisdom, Words of Advice*, ed. Krasner); documentación (Adele y Robson, tres libros, publicaron el tercero).

- El *Byte* de agosto 1981 (tapa del globo aerostático) generó una avalancha de pedidos de compra mundiales.
- Más decepciones: **Twinkle** (máquina Smalltalk-80 basada en 68000, 1982) ignorada; versión para el Star (“Molasses”) en Mesa; el sistema **Analyst** usado en producción **en la CIA** sin que Xerox invirtiera.
- Recién en 1988 se fundó **ParcPlace Systems** (Adele CEO 1988–92); primer Smalltalk reutilizable y totalmente compilado en 11 plataformas (rediseño de la VM por Peter Deutsch). Se agregaron **closures de bloques, manejo de excepciones**, bibliotecas y puentes a C/bases de datos.
- En 1993, ANSI creó el comité X3J20 (estándar Smalltalk). “*Smalltalk sigue siendo un objeto sorprendentemente persistente.*”

Nota técnica — Máquina virtual (VM) y byte-codes

Smalltalk-80 se especificó como una **máquina virtual**: un intérprete de byte-codes portable. Cada fabricante implementaba esa VM en su hardware, así el mismo Smalltalk corría en muchas máquinas distintas — el mismo modelo que después popularizó Java (“compilar una vez, correr en cualquier lado”).

12 Sesión de preguntas y respuestas (lo destacado)

- **¿Futuro de Smalltalk vs C++?** Kay: los lenguajes deberían “explotar” cada 20 años porque cuando duran demasiado **contaminan ideas futuras**. No ve correlación entre cuánta gente usa un lenguaje y cuán bueno es. Lo más importante de Smalltalk/LISP: la **facilidad de meta-definición** — no tomar el lenguaje como features de un diseñador, sino como una manera de **habilitar el lenguaje que realmente necesitás**.
- Adele: los lenguajes se diseñan para propósitos específicos. Hace falta facilitar que la gente que conoce su dominio **invente su propio lenguaje**.
- **¿OOP con miles de partes vs OOP para chicos — relacionadas?** Kay: cada nuevo medio trae modos de pensamiento. Los modos falaces sobre la computación tienen que ver con **sistemas complejos de muchas partes interactuando**; darles a los chicos una forma de atacar la complejidad los mete en un espacio de pensamiento más interesante.
- **¿Las “D machines” ayudaron o perjudicaron a Smalltalk?** Kay: el atraso de la primera D machine **perjudicó** a Smalltalk; incluso **regresó hacia formas tipo Simula por eficiencia**. Debieron seguir construyendo máquinas más rápidas hasta entender bien la OOP.

13 Glosario rápido de tecnologías y personas

Tecnologías / sistemas

Nombre	Qué es / por qué importa
Burroughs B5000 (1961)	Mainframe de Bob Barton; byte-codes, memoria virtual segmentada, descriptors (referencias seguras), PRT, protección tipo capability. Fuente conceptual mayor de Smalltalk.
Sketchpad (1963)	Primer programa de gráficos interactivos (Sutherland); masters/instances, restricciones, ventanas con clipping/zoom.
Simula I / 67	Lenguajes de simulación noruegos; clases, instancias con estado propio y (Simula-67) la herencia. Ancestro directo de OOP.
LISP (1960)	Lenguaje de McCarthy; intérprete metacircular (~1 página). Modelo de elegancia y meta-definición.
JOSS	Sistema interactivo de RAND; estándar de buena interacción con el usuario final.
NLS (Engelbart)	“oN-Line System”; hipertexto, mouse, colaboración. La “augmentación del intelecto”.
GRAIL (RAND)	Interfaz gráfica con tableta y reconocimiento de gestos; manipulación directa, sin modos.
LOGO (Papert)	Lenguaje para enseñar a programar a chicos (las “tortugas”). Influencia pedagógica clave.
FLEX (Kay-Cheadle)	Primer intento de PC orientada a objetos (1967–69); precursor de Smalltalk.
LINC (Wes Clark, 1962)	Considerada la primera PC real.
ALTO (Thacker, 1973)	El “Interim Dynabook”; bitmap, ventanas, mouse, Ethernet. Plataforma de Smalltalk.
NoteTaker (≈1978)	“Laptop” de PARC basada en Intel 8086; corrió Smalltalk en un avión.
Dorado	“D machine”, sucesor rápido del ALTO; usado en la demo a Apple.
OOZE	Sistema de memoria virtual por objetos de Smalltalk-74/76.
Ethernet	LAN por conmutación de paquetes inventada en PARC (Metcalfe).
Xerox Star (1981)	Primer producto comercial con GUI (no exitoso).
Lisa / Macintosh	Llevaron la GUI de PARC al mercado masivo.
bitblt	Operación de copia de bloques de píxeles; base de los gráficos 2D.
byte-code / VM	Instrucciones compactas interpretadas por una máquina virtual portable.

Personas

- **Alan Kay** — instigador y diseñador original; concibió el Dynabook y la computación personal como “medio dinámico”.
- **Dan Ingalls** — implementador central; creador de Smalltalk-76; “lo hacés y ya está”.
- **Adele Goldberg** — enseñanza con chicos, design templates, documentación, release de Smalltalk-80, luego CEO de ParcPlace.

- **Bob Barton** — diseñó el B5000; “las partes deben tener el mismo poder que el todo”.
- **Ivan Sutherland** — Sketchpad; primer jefe de ARPA-IPTO joven.
- **Dave Evans** — mentor de Kay en Utah; “siempre dejá un bit extra”.
- **Douglas Engelbart** — NLS, el mouse, la augmentación del intelecto.
- **Seymour Papert / Marvin Minsky** — pedagogía (LOGO, Piaget), modos de pensar la complejidad.
- **Butler Lampson / Chuck Thacker** — propusieron y construyeron el ALTO; OS, transacciones, “Huffman pobre”.
- **Larry Tesler** — miniMOUSE (WYSIWYG modeless), browsers de Smalltalk-76, herencia por delegación; luego Apple.
- **Ted Kaehler** — tortugas OO, TWANG, system tracer, OOZE.
- **Peter Deutsch** — LISP para PDP-1 a los 15; rediseño de la VM en ParcPlace.
- **Bob Taylor** — armó el Computer Science Lab de PARC.

14 Cierre — 5 ideas para llevarse al final

1. **Smalltalk es una recursión sobre la noción de computadora:** todo es un objeto, los objetos se comunican solo por mensajes, cada uno encapsula estado + proceso (mónada / célula).
2. **Las influencias clave fueron:** el sistema de archivos del B220, el B5000 (Barton), Sketchpad (Sutherland), Simula (clases/instancias/herencia), LISP (meta-definición), JOSS/GRAIL/LOGO (interacción y pedagogía), FLEX (Kay) y el **CDL de Fisher** (control). Todo venía del clima de **ARPA** y se cristalizó en **Xerox PARC**.
3. **OOP, según Kay, no es “ADT con sintaxis linda”:** su poder real está en cuatro técnicas juntas (estado persistente, polimorfismo, instanciación, métodos-como-metas) y, sobre todo, en el **late binding** y la **protección total** de objetos de primera clase.
4. **Línea evolutiva:** Smalltalk-71 (parser/pattern-matching) → **Smalltalk-72** (primer sistema real, 6 principios, polimorfismo, en el ALTO) → **Smalltalk-76** (intensional, herencia tipo Simula, sintaxis keyword/operador, OOZE, ~180x más rápido) → **Smalltalk-80** (release multivendor vía máquina virtual: Apple/HP/DEC/Tektronix; ASCII, blocks tipo lambda, metaclasses).
5. **El meta-relato:** el proyecto era sobre **personas, cultura y visión** (financiar gente y direcciones, contratar ingenuos llenos del sueño, “inventar el futuro”) y sobre el aprendizaje de los chicos como brújula — más que sobre features de un lenguaje. Smalltalk inventó o consolidó la PC, las ventanas superpuestas, la GUI moderna y la OOP, aunque Xerox no supo comercializarlo.